

Exercice 03 — TEST-6 a TEST-8 : Donnees dynamiques, mocking et pipeline CI pour la plateforme

Projet fil rouge : Code un max - Suite de tests automatisés de la plateforme de cours

Cet exercice **s'ajoute au travail précédent**. Vous continuez à construire le projet étape par étape.

Dans cet exercice : La suite E2E en Page Object Model construite au chapitre 2 (login, catalogue, inscription, tableau de bord) fonctionne en local mais utilise des comptes ecrits en dur et depend du service de paiement externe de la plateforme. L'equipe qualite de Code un max veut des tests reproductibles et une execution automatique a chaque push.

Avant de commencer

L'**Exemple 03** ([exemples/exemple-03/exemple-03.md](#)) t'a fait pratiquer la technique sur un cas neutre. Ici, tu l'appliques au projet **Code un max - Suite de tests automatisés de la plateforme de cours**. Assure-toi de l'avoir lu et suivi avant de commencer.

Vous devez aussi avoir **terminé l'Exercice 02**, car cet exercice s'appuie sur le travail déjà réalisé.

Objectif

Rendre la suite de tests de la plateforme Code un max independante des donnees et des services externes, puis l'automatiser dans GitHub Actions a chaque commit.

Prérequis

- Python 3.11+ installé
- Git
- VS Code
- Chrome ou Firefox
- Compte GitHub
- Bases de la programmation Python
- Notions de développement web
- Avoir complété les exercices précédents du projet fil rouge

Scénario

Tu reprends le depot du fil rouge. On te confie les tickets TEST-6 (donnees Faker), TEST-7 (mock paiement) et TEST-8 (pipeline CI) : introduire des donnees generees dynamiquement, isoler l'appel au service de paiement via mocking, gerer plusieurs environnements (local et CI) et livrer un pipeline GitHub Actions avec trois jobs. Le rapport de tests doit etre recupere en artifact.

Étapes à réaliser

Étape 1 — Generer les comptes apprenants avec Faker

Remplacer les identifiants écrits en dur dans les tests d'inscription par des données générées.

Ce que vous devez faire :

- Créer une fixture qui produit un apprenant (nom, email, mot de passe) via Faker en locale fr_FR
- Utiliser un seed pour que les runs soient reproductibles
- Brancher cette fixture sur le test d'inscription existant

Résultat attendu : Le test d'inscription utilise un apprenant généré à chaque exécution, sans email codé en dur.

Piste : Un provider Faker `profile()` donne un dictionnaire complet, mais tu peux aussi composer `name()`, `email()` et `password()`.

Erreur fréquente : Générer un email déjà présent en base : préfixe-le avec un timestamp ou uuid pour garantir l'unicité.

Étape 2 — Isoler les données entre les tests

Garantir qu'un test ne laisse pas de compte ou d'inscription qui pollue le test suivant.

Ce que vous devez faire :

- Mettre la fixture apprenant en scope 'fonction' pour repartir propre à chaque test
- Ajouter un nettoyage (teardown) qui supprime le compte créé après le test
- Vérifier qu'un test peut être relancé deux fois de suite sans conflit

Résultat attendu : Deux exécutions consécutives de la suite passent sans erreur de type 'email déjà utilisé'.

Piste : Le `yield` dans une fixture pytest sépare le setup (avant) du teardown (après `yield`).

Étape 3 — Mocker le service de paiement d'inscription

L'inscription a un cours payant appelé une API de paiement externe. On veut la simuler pour ne pas dépendre du fournisseur.

Ce que vous devez faire :

- Identifier l'appel HTTP vers l'API de paiement dans le code de la plateforme
- Utiliser la bibliothèque `responses` pour renvoyer un paiement accepté (status 200)
- Ajouter un second test qui simule un paiement refusé (status 402) et vérifie le message d'erreur affiché

Résultat attendu : Les tests d'inscription payante passent sans appeler le vrai service, y compris le cas refus.

Piste : Le cas d'échec est souvent plus révélateur que le cas nominal : vérifie que la plateforme gère bien un refus.

Erreur fréquente : Oublier `@responses.activate` : les vrais appels partent alors sur le réseau et le test devient instable.

Étape 4 — Configurer les environnements local et CI

L'URL de base et les identifiants changent entre la machine locale et la CI.

Ce que vous devez faire :

- Créer un fichier `.env` pour les variables locales (`BASE_URL`, comptes de service)
- Lire ces variables dans une fixture de configuration `pytest`
- Ajouter `.env` au `.gitignore` et documenter les variables attendues dans un `.env.example`

Résultat attendu : La suite lit `BASE_URL` depuis l'environnement, avec une valeur par défaut locale si la variable est absente.

Piste : `python-dotenv` charge le `.env` ; en CI tu passeras les valeurs via les variables du workflow, pas via un fichier.

Étape 5 — Ecrire le pipeline GitHub Actions

Créer `.github/workflows/ci.yml` avec trois jobs distincts.

Ce que vous devez faire :

- Job `lint` : lancer `ruff` sur le code
- Job `unit_tests` : lancer les tests unitaires et mockes (rapides, sans navigateur)
- Job `e2e_tests` : lancer les tests E2E (peut dépendre du job `unit_tests`)
- Déclencher le workflow sur `push` et `pull_request`

Résultat attendu : Les trois jobs apparaissent dans l'onglet Actions et s'exécutent à chaque `push`.

Piste : Utilise `'needs:'` pour enchaîner `e2e_tests` après `unit_tests` et ne pas gaspiller de minutes si l'unitaire échoue.

Erreur fréquente : Lancer les tests E2E sans navigateur installé : ajoute l'installation de Chrome/`chromedriver` dans le job `e2e`.

Étape 6 — Publier le rapport `pytest` en artifact

Rendre le résultat des tests téléchargeable depuis GitHub après chaque run.

Ce que vous devez faire :

- Générer un rapport HTML avec `pytest-html` dans le job de tests
- Uploader ce rapport avec `actions/upload-artifact`
- S'assurer que l'upload a lieu même si des tests échouent

Résultat attendu : Un artifact `'rapport-tests'` contenant le HTML est disponible dans le run, même en cas d'échec.

Piste : `if: always()` sur le step d'upload garantit la récupération du rapport quand un test casse.

Vérification

Quand vous avez terminé, vérifiez que :

- ☐ La suite passe deux fois de suite en local sans conflit de données
 - ☐ Aucun test unitaire ne sort sur le réseau (vérifiable en coupant la connexion)
 - ☐ Un push déclenche automatiquement les trois jobs sur GitHub
 - ☐ Le rapport HTML est téléchargeable en artifact après le run
-

En pratique

Le piège que je vois sur ce ticket : les étudiants mockent tout et perdent la confiance dans la suite. Garde au moins un test E2E qui parle à une vraie instance de staging. Le mock est parfait pour l'unitaire et les cas d'erreur difficiles à reproduire, pas pour prouver que le parcours d'inscription marche de bout en bout.

Bonus (Facultatif mais Recommandé)

Ajoute un badge de statut CI dans le README du dépôt et fais échouer volontairement un test pour vérifier que le pipeline passe au rouge et que le rapport reste disponible.

Livrable attendu

Dépôt avec fixtures Faker, tests mockés du paiement, gestion `.env/.env.example`, `.github/workflows/ci.yml` (lint + unit_tests + e2e_tests) et artifact de rapport, commit référence TEST-6 à TEST-8.

Une fois terminé, consultez la **correction** dans [corrections-exercices/correction-exercice-03/correction-exercice-03.md](#) pour comparer votre travail.